

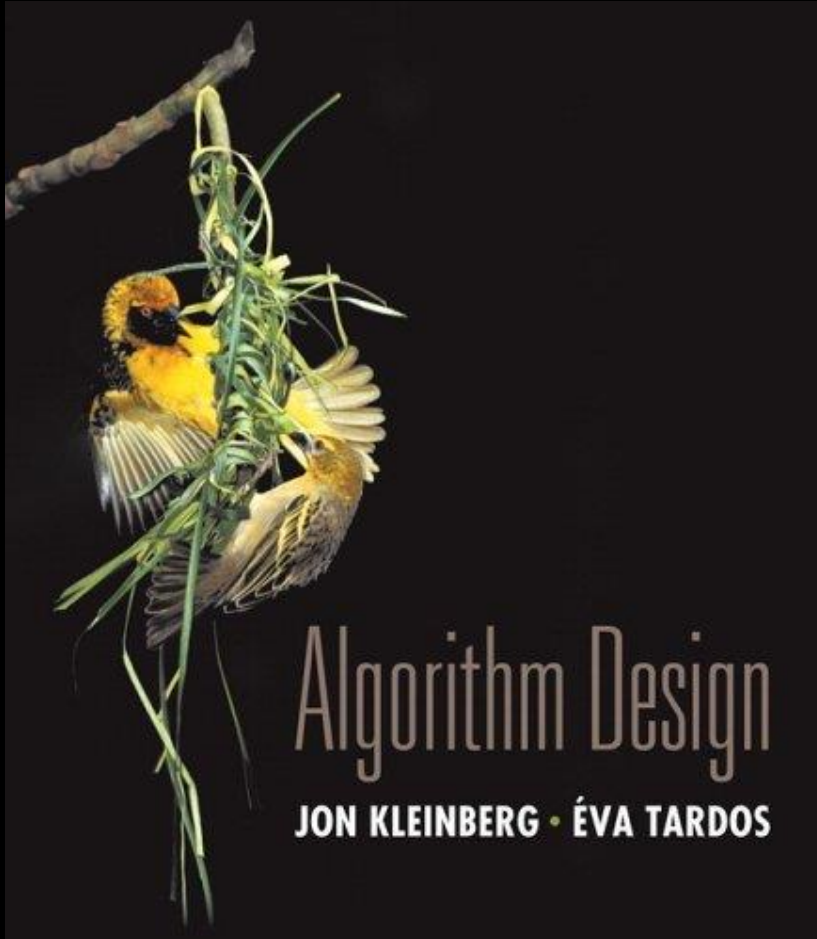
Course evaluation

<https://evaluation.shanghaitech.edu.cn/tqnmaep/login>



Chapter 11

Approximation Algorithms

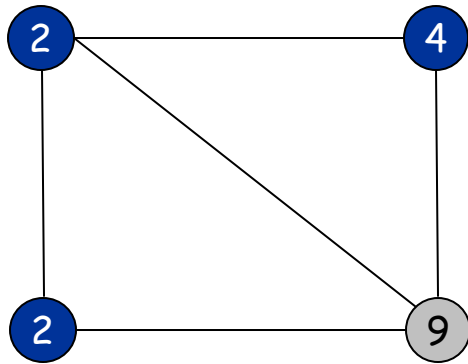


Slides by Kevin Wayne.
Copyright © 2005 Pearson-Addison Wesley.
All rights reserved.

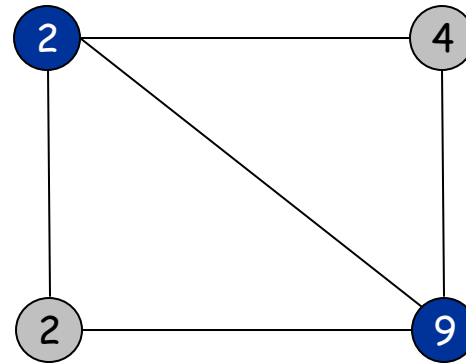
11.4 The Pricing Method: Vertex Cover

Weighted Vertex Cover

Weighted vertex cover. Given a graph G with vertex weights, find a vertex cover of minimum weight.



$$\text{weight} = 2 + 2 + 4$$



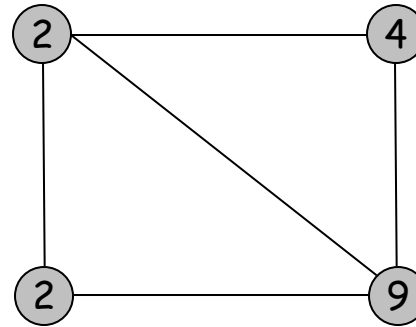
$$\text{weight} = 11$$

Weighted Vertex Cover

Pricing method. Each edge must be covered by some vertex. Edge e pays price $p_e \geq 0$ to use a vertex.

Fairness. Edges incident to vertex i should pay $\leq w_i$ in total.

for each vertex i : $\sum_{e=(i,j)} p_e \leq w_i$



Fairness Lemma. For any vertex cover S and any fair prices p_e :
 $\sum_e p_e \leq w(S).$

Proof.

$$\sum_{e \in E} p_e \leq \sum_{i \in S} \sum_{e=(i,j)} p_e \leq \sum_{i \in S} w_i = w(S). \quad \blacksquare$$

each edge e covered by
at least one node in S

sum fairness inequalities
for each node in S

Pricing Method

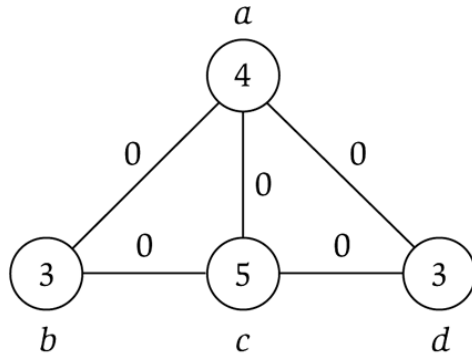
Pricing method. Set prices and find vertex cover simultaneously.

```
Weighted-Vertex-Cover-Approx(G, w) {  
  foreach e in E  
    pe = 0  
  
  while (∃ edge i-j such that neither i nor j are tight)  
    select such an edge e  
    increase pe without violating fairness  
}  
  
S ← set of all tight nodes  
return S  
}
```

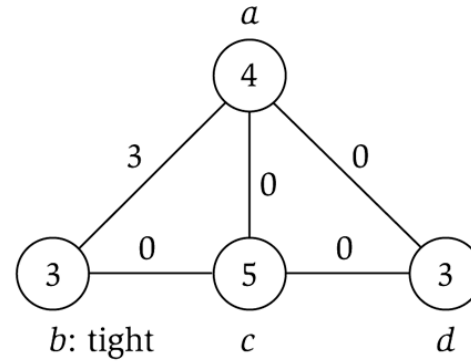
$$\sum_{e=(i,j)} p_e = w_i$$

↓

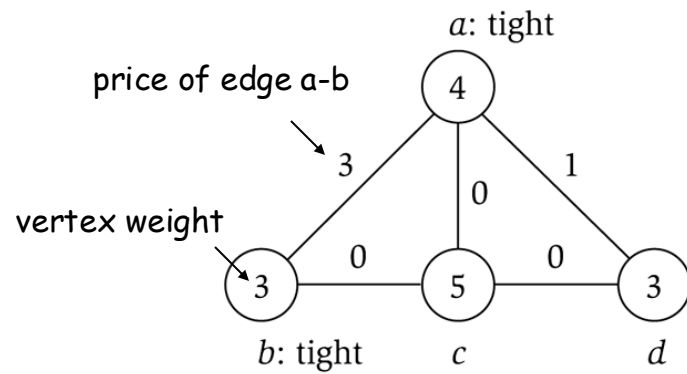
Pricing Method



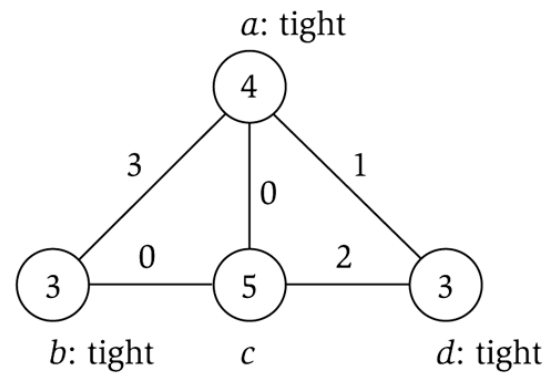
(a)



(b)



(c)



(d)

Pricing Method: Analysis

Theorem. Pricing method is a 2-approximation.

Pf.

- Algorithm terminates since at least one new node becomes tight after each iteration of while loop.
- Let S = set of all tight nodes upon termination of algorithm. S is a vertex cover: if some edge i - j is uncovered, then neither i nor j is tight. But then while loop would not terminate.
- Let S^* be optimal vertex cover. We show $w(S) \leq 2w(S^*)$.

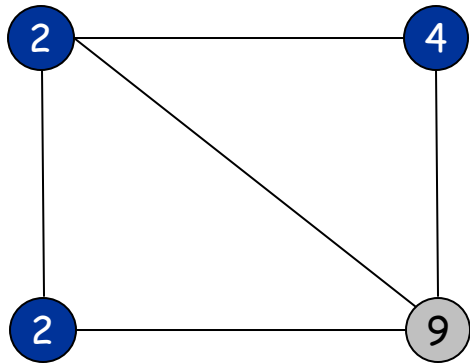
$$w(S) = \sum_{i \in S} w_i = \sum_{i \in S} \sum_{e=(i,j)} p_e \leq \sum_{i \in V} \sum_{e=(i,j)} p_e = 2 \sum_{e \in E} p_e \leq 2w(S^*). \quad \blacksquare$$

$\uparrow$ all nodes in S are tight $\uparrow$ $S \subseteq V$, prices ≥ 0 $\uparrow$ each edge counted twice $\uparrow$ fairness lemma

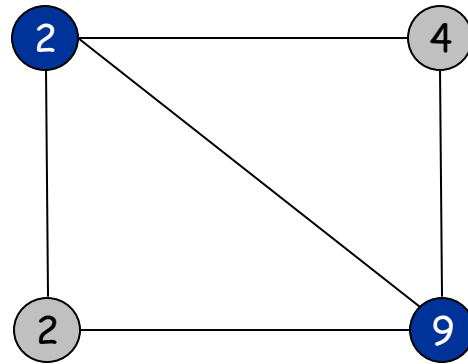
11.6 LP Rounding: Vertex Cover

Weighted Vertex Cover

Weighted vertex cover. Given an undirected graph $G = (V, E)$ with vertex weights $w_i \geq 0$, find a minimum weight subset of nodes S such that every edge is incident to at least one vertex in S .



weight = $2 + 2 + 4$



weight = 11

Weighted Vertex Cover: IP Formulation

Weighted vertex cover. Given an undirected graph $G = (V, E)$ with vertex weights $w_i \geq 0$, find a minimum weight subset of nodes S such that every edge is incident to at least one vertex in S .

Integer programming formulation.

- Model inclusion of each vertex i using a 0/1 variable x_i .

$$x_i = \begin{cases} 0 & \text{if vertex } i \text{ is not in vertex cover} \\ 1 & \text{if vertex } i \text{ is in vertex cover} \end{cases}$$

Vertex covers in 1-1 correspondence with 0/1 assignments:

$$S = \{i \in V : x_i = 1\}$$

- Objective function: minimize $\sum_i w_i x_i$.
- For edge (i,j) , must take either i or j (or both): $x_i + x_j \geq 1$.

Weighted Vertex Cover: IP Formulation

Weighted vertex cover. Integer programming formulation.

$$\begin{array}{ll} (ILP) \min & \sum_{i \in V} w_i x_i \\ \text{s. t.} & x_i + x_j \geq 1 \quad (i, j) \in E \\ & x_i \in \{0, 1\} \quad i \in V \end{array}$$

Observation. If x^* is optimal solution to (ILP), then $S = \{i \in V : x_i^* = 1\}$ is a min weight vertex cover.

Integer Programming

INTEGER-PROGRAMMING. Given constant integers c_j , b_i , a_{ij} , find integers x_j that satisfy:

$$\begin{array}{ll} \max & c^t x \\ \text{s. t.} & Ax \geq b \\ & x \text{ integral} \end{array}$$

Observation. Vertex cover formulation proves that integer programming is NP-hard search problem.

↑
even if all coefficients are 0/1 and
at most two variables per inequality

Linear Programming

Linear programming. Max/min linear objective function subject to linear inequalities.

- Input: integers c_j, b_i, a_{ij} .
- Output: **real numbers** x_j .

$$\begin{array}{ll} (LP) & \max \quad c^t x \\ & \text{s. t.} \quad Ax \geq b \\ & \quad \quad x \geq 0 \end{array}$$

Linear. No x^2 , xy , $\arccos(x)$, $x(1-x)$, etc.

Simplex algorithm. [Dantzig 1947] Can solve LP in practice.

Ellipsoid algorithm. [Khachian 1979] Can solve LP in poly-time.

Weighted Vertex Cover: LP Relaxation

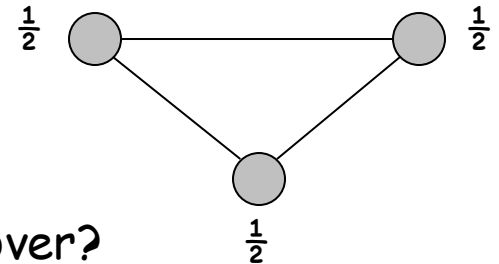
Weighted vertex cover. Linear programming formulation.

$$\begin{array}{ll} (LP) \min & \sum_{i \in V} w_i x_i \\ \text{s. t.} & x_i + x_j \geq 1 \quad (i, j) \in E \\ & x_i \geq 0 \quad i \in V \end{array}$$

Observation. Optimal value of (LP) is $\leq$ optimal value of (ILP).

Pf. LP has fewer constraints.

Note. LP is not equivalent to vertex cover.



Q. How can solving LP help us find a small vertex cover?

A. Solve LP and **round** fractional values.

Weighted Vertex Cover

Theorem. If x^* is optimal solution to (LP), then $S = \{i \in V : x_i^* \geq \frac{1}{2}\}$ is a vertex cover whose weight is at most twice the min possible weight.

Pf.

[S is a vertex cover]

- Consider an edge $(i, j) \in E$.
- Since $x_i^* + x_j^* \geq 1$, either $x_i^* \geq \frac{1}{2}$ or $x_j^* \geq \frac{1}{2} \Rightarrow (i, j)$ covered.

[S has desired cost]

- Let S^* be optimal vertex cover. Then

$$\sum_{i \in S^*} w_i \geq \sum_{i \in V} w_i x_i^* \geq \sum_{i \in S} w_i x_i^* \geq \frac{1}{2} \sum_{i \in S} w_i$$

LP is a relaxation $x_i^* \geq \frac{1}{2}$

Weighted Vertex Cover

Theorem. 2-approximation algorithm for weighted vertex cover.

Theorem. [Dinur-Safra 2001] If $P \neq NP$, then no ρ -approximation for $\rho < 1.3607$, even with unit weights.


$$10\sqrt{5} - 21$$

Open research problem. Close the gap.

11.8 Knapsack Problem

Polynomial Time Approximation Scheme

PTAS. $(1 + \varepsilon)$ -approximation algorithm for any constant $\varepsilon > 0$.

Consequence. PTAS produces arbitrarily high quality solution, but trades off accuracy for time.

This section. PTAS for knapsack problem via rounding and scaling.

Knapsack Problem

Knapsack problem.

- Given n objects and a "knapsack."
- Item i has value $v_i > 0$ and weighs $w_i > 0$. ← we'll assume $w_i \leq W$
- Knapsack can carry weight **up to** W .
- Goal: fill knapsack so as to maximize total value.

Ex: $\{ 3, 4 \}$ has value 40.

$$W = 11$$

Item	Value	Weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

Knapsack is NP-Complete

KNAPSACK: Given a finite set X , nonnegative weights w_i , nonnegative values v_i , a weight limit W , and a target value V , is there a subset $S \subseteq X$ such that:

$$\begin{aligned}\sum_{i \in S} w_i &\leq W \\ \sum_{i \in S} v_i &\geq V\end{aligned}$$

SUBSET-SUM: Given a finite set X , nonnegative values u_i , and an integer U , is there a subset $S \subseteq X$ whose elements sum to exactly U ?

Claim. $\text{SUBSET-SUM} \leq_p \text{KNAPSACK}$.

Pf. Given instance $(u_1, \dots, u_n, U)$ of SUBSET-SUM, create KNAPSACK instance:

$$\begin{aligned}v_i = w_i = u_i \quad \sum_{i \in S} u_i &\leq U \\ V = W = U \quad \sum_{i \in S} u_i &\geq U\end{aligned}$$

Knapsack Problem: Dynamic Programming 1

Def. $OPT(i, w)$ = max value subset of items $1, \dots, i$ with weight limit w .

- Case 1: OPT does not select item i .
 - OPT selects best of $1, \dots, i-1$ using up to weight limit w
- Case 2: OPT selects item i .
 - new weight limit = $w - w_i$
 - OPT selects best of $1, \dots, i-1$ using up to weight limit $w - w_i$

$$OPT(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ OPT(i-1, w) & \text{if } w_i > w \\ \max \{ OPT(i-1, w), v_i + OPT(i-1, w - w_i) \} & \text{otherwise} \end{cases}$$

Running time. $O(n W)$.

- W = weight limit.
- **Not polynomial** in input size!

Knapsack Problem: Dynamic Programming II

Def. $OPT(i, v)$ = min weight subset of items 1, ..., i that yields value **exactly** v.

- Case 1: OPT does not select item i.
 - OPT selects best of 1, ..., i-1 that achieves exactly value v
- Case 2: OPT selects item i.
 - consumes weight w_i , new value needed = $v - v_i$
 - OPT selects best of 1, ..., i-1 that achieves exactly value v

$$OPT(i, v) = \begin{cases} 0 & \text{if } v = 0 \\ \infty & \text{if } i = 0, v > 0 \\ OPT(i-1, v) & \text{if } v_i > v \\ \min \{ OPT(i-1, v), w_i + OPT(i-1, v - v_i) \} & \text{otherwise} \end{cases}$$

Running time. $O(n V^*) = O(n^2 v_{\max})$.

- V^* = maximum possible total value $\leq n v_{\max}$
- **Not polynomial** in input size!

Knapsack: PTAS

Intuition for approximation algorithm.

- Round all values up to lie in smaller range.
- Run dynamic programming algorithm on rounded instance.
- Return optimal items in rounded instance.

Item	Value	Weight
1	1,342,211	1
2	6,563,429	2
3	18,100,134	5
4	22,217,800	6
5	28,343,199	7

$W = 11$

original instance



Item	Value	Weight
1	2	1
2	7	2
3	19	5
4	23	6
5	29	7

$W = 11$

rounded instance

Knapsack: PTAS

Knapsack PTAS. Round up all values:

- $v_{\max}$ = largest value in original instance
- ε = precision parameter
- θ = scaling factor = $\varepsilon v_{\max} / 2n$

$$\bar{v}_i = \left\lceil \frac{v_i}{\theta} \right\rceil \theta \quad \hat{v}_i = \left\lceil \frac{v_i}{\theta} \right\rceil$$

Observation. Optimal solution to problems with $\bar{v}$ or $\hat{v}$ are equivalent.

Intuition. $\bar{v}$ close to v so optimal solution using $\bar{v}$ is nearly optimal;
 $\hat{v}$ small and integral so dynamic programming algorithm is fast.

Running time. $O(n^3 / \varepsilon)$.

- Dynamic program II running time is $O(n^2 \hat{v}_{\max})$, where

$$\hat{v}_{\max} = \left\lceil \frac{v_{\max}}{\theta} \right\rceil = \left\lceil \frac{2n}{\varepsilon} \right\rceil$$

Knapsack: PTAS

Knapsack PTAS. Round up all values: $\bar{v}_i = \left\lceil \frac{v_i}{\theta} \right\rceil \theta$

Theorem. If S is solution found by our algorithm and S^* is any other feasible solution then $(1+\varepsilon) \sum_{i \in S} v_i \geq \sum_{i \in S^*} v_i$ for $\varepsilon \leq 1$

Pf. Let S^* be any feasible solution satisfying weight constraint.

$$\sum_{i \in S^*} v_i \leq \sum_{i \in S^*} \bar{v}_i \quad \text{always round up}$$

$$\leq \sum_{i \in S} \bar{v}_i \quad \text{solve rounded instance optimally}$$

$$\leq \sum_{i \in S} (v_i + \theta) \quad \text{never round up by more than } \theta$$

$$\leq \sum_{i \in S} v_i + n\theta \quad |S| \leq n$$

$$= \sum_{i \in S} v_i + \frac{1}{2} \varepsilon v_{\max} \quad \theta = \varepsilon v_{\max} / 2n$$

$$\leq (1+\varepsilon) \sum_{i \in S} v_i \quad v_{\max} \leq 2 \sum_{i \in S} v_i$$

Choosing $S^* = \{ \max \}$

$$v_{\max} \leq \sum_{i \in S} v_i + \frac{1}{2} \varepsilon v_{\max}$$

$$\leq \sum_{i \in S} v_i + \frac{1}{2} v_{\max}$$

Thus

$$v_{\max} \leq 2 \sum_{i \in S} v_i$$

Chapter Summary

Approximation Algorithms

To solve an NP-hard problem, must sacrifice one of three desired features.

- Solve problem to optimality.
- Solve problem in poly-time.
- Solve arbitrary instances of the problem.

ρ -approximation algorithm.

- Guaranteed to run in poly-time.
- Guaranteed to solve arbitrary instance of the problem
- Guaranteed to find solution within ratio ρ of true optimum.

Outline

Example problems

- Load Balancing
 - Greedy algorithm is a $3/2$ -approximation
- Center Selection
 - Greedy algorithm is a 2-approximation
- Weighted Vertex Cover
 - Pricing method is a 2-approximation
 - Linear programming + rounding is a 2-approximation
- Knapsack Problem
 - Polynomial Time Approximation Scheme: $(1 + \varepsilon)$ -approximation algorithm for any constant $\varepsilon > 0$
 - Rounding and scaling